30/10/26 Week-4

Cuckoo search

Application: Traffic Signal Timing Optimization

Algorithm:

Pseudo code:

BEGIN

1. Initialize population of nests
2. Generate random green signal timings
3. Calculate fitness using:
   - Vehicle waiting time
   - Queue length
   - Vehicle arrival rate
4. REPEAT for each iteration:
   - FOR each nest:
     - Generate a new solution using Levy Flight
     - Calculate its Fitness
     - IF new solution is better:
       - Replace the old nest.
   - END FOR
   - Abandon some poor nests
   - Generate new random solutions.
   - Update the best solution
5. Display optimal green signal timings.

END

o/p:

Enter numbers of traffic directions: 4

| Direction name | vehicle arrival rate | queue length | avg waiting ti |
|---|---|---|---|
| North | 20 | 30 | 2 |
| South | 15 | 20 | 3 |
| East | 25 | 40 | 4 |
| West | 10 | 15 | 2 |

Page No.
Date:

Enter total signal cycle time : 120
--- OPTIMIZED SIGNAL TIMINGS ---
North : 27.5 seconds
South : 28.52 seconds
East : 44.01 seconds
West : 19.98 seconds
Total cycle Time : 120.0 seconds
Minimum Congestion Fitness : 1827.19
Signal timing optimization completed!

8/11/25

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wlW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] 1m

```
import random

# Traffic Signal Optimization using Cuckoo Search

print("TRAFFIC SIGNAL TIMING OPTIMIZATION")
print("----------------------------------")

# User input
n = int(input("Enter number of traffic directions: "))

directions = []
arrival = []
queue = []
waiting = []

for i in range(n):

    print("\nDirection", i + 1)
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:57 06-11-2026

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wlW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] 1m

```
    return total


# Generate valid signal timings
def normalize(solution):

    solution = [
        max(min_green, min(x, cycle))
        for x in solution
    ]

    available = cycle - n * min_green

    extra = [max(0, x - min_green) for x in solution]
    total = sum(extra)

    if total == 0:
        return [cycle / n] * n

    return [
        min_green + (x / total) * available
        for x in extra
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:57 06-11-2026

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wIW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] 1m

```
    ]


# Levy flight
def levy_flight(solution):

    new_solution = []

    for x in solution:
        step = random.gauss(0, 1)

        new_solution.append(x + step * 5)

    return normalize(new_solution)


# Initialize nests
population = []

for _ in range(nests):

    solution = [
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:57 06-11-2026

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wlW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] ✓ 1m

```
if max_green < min_green * 0 + 10:
    raise ValueError("Cycle time is too small for the number of directions.")

# Fitness function
def fitness(solution):

    total = 0

    for i in range(n):

        # Estimated waiting time after signal allocation
        effective_wait = waiting[i] * (
            cycle / (solution[i] + 1)
        )

        # Congestion score
        total += (
            arrival[i] * effective_wait
            + queue[i] * effective_wait
        )

    return total
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:57 06-11-2026

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wIW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] 1m

```
        name = input("Enter direction name: ")
        directions.append(name)

        a = float(input("Enter vehicle arrival rate (vehicles/min): "))
        q = int(input("Enter queue length (vehicles): "))
        w = float(input("Enter average waiting time (minutes): "))

        arrival.append(a)
        queue.append(q)
        waiting.append(w)

# Signal cycle duration
cycle = float(input("\nEnter total signal cycle time (seconds): "))

# Cuckoo Search parameters
nests = 20
iterations = 100
pa = 0.25

min_green = 10
max_green = cycle - min_green * (n - 1)
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:57 06-11-2026

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wIW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] 1m

```
        solution = [
            random.uniform(min_green, cycle / n)
            for _ in range(n)
        ]

        population.append(normalize(solution))


    # Cuckoo Search
    best_solution = None
    best_fitness = float("inf")

    for iteration in range(iterations):

        for i in range(nests):

            new_solution = levy_flight(population[i])

            new_fitness = fitness(new_solution)

            old_fitness = fitness(population[i])
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:58 06-11-2026

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wlW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] 1m

```
        if new_fitness < old_fitness:
            population[i] = new_solution

    # Abandon worst nests
    population.sort(key=fitness)

    for i in range(int(pa * nests), nests):

        population[i] = normalize([
            random.uniform(min_green, cycle / n)
            for _ in range(n)
        ])

    # Update best solution
    current_best = min(population, key=fitness)

    if fitness(current_best) < best_fitness:
        best_fitness = fitness(current_best)
        best_solution = current_best[:]
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:58 06-11-2026

Bio Inspire | week3_1BM | Untitled5.ipynb | week4_1BM | BIS_5TH_SE | BIS_5TH_SE | (no subject) | New Tab

colab.research.google.com/drive/1G2VxS1Zy9H1CMpm7l6AgNwM9D0UG4wlW#scrollTo=9lQFgJg97ImV

School

week4_1BM24CS104.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[1] 1m

```
# Display output
print("\n========== OPTIMIZED SIGNAL TIMINGS ==========")

for i in range(n):

    print(directions[i], ":", round(best_solution[i], 2),
          "seconds")

print("\nTotal Cycle Time:",
      round(sum(best_solution), 2), "seconds")

print("Minimum Congestion Fitness:",
      round(best_fitness, 2))

print("\nSignal timing optimization completed!")
```

```
TRAFFIC SIGNAL TIMING OPTIMIZATION
----------------------------------
Enter number of traffic directions: 4

Direction 1
```

Variables Terminal

1:49 PM Python 3

ENG IN 50% 13:58 06-11-2026

```
Direction 4
Enter direction name: West
Enter vehicle arrival rate (vehicles/min): 10
Enter queue length (vehicles): 15
Enter average waiting time (minutes): 2

Enter total signal cycle time (seconds): 120

========== OPTIMIZED SIGNAL TIMINGS ==========
North : 27.5 seconds
South : 28.52 seconds
East : 44.01 seconds
West : 19.98 seconds

Total Cycle Time: 120.0 seconds
Minimum Congestion Fitness: 1827.19

Signal timing optimization completed!
```

```
Direction 1
Enter direction name: North
Enter vehicle arrival rate (vehicles/min): 20
Enter queue length (vehicles): 30
Enter average waiting time (minutes): 2

Direction 2
Enter direction name: South
Enter vehicle arrival rate (vehicles/min): 15
Enter queue length (vehicles): 20
Enter average waiting time (minutes): 3

Direction 3
Enter direction name: East
Enter vehicle arrival rate (vehicles/min): 25
Enter queue length (vehicles): 40
Enter average waiting time (minutes): 4

Direction 4
Enter direction name: West
Enter vehicle arrival rate (vehicles/min): 10
Enter queue length (vehicles): 15
Enter average waiting time (minutes): 2
```